JAVA PROGRAMMING — UNIT 1

Handwritten-style class notes | BCA | Simple English

1. History and Evolution of Java

Java is a high-level, class-based and object-oriented programming language. It was developed by James Gosling and his team at Sun Microsystems.

- *The Java project started in 1991 and was first named Oak.*
- *The name was changed to Java, and Java was officially announced in 1995.*
- *The first public release, Java 1.0, came in 1996.*
- *Oracle acquired Sun Microsystems in 2010 and now stewards Java.*

Important milestones

Year	Development
1991	Java project started; original name was Oak.
1995	Java officially announced.
1996	Java 1.0 released.
1998	Java 2 introduced.
2004	Java 5 introduced important language improvements.
2014	Java 8 introduced major features such as lambda expressions.
2021 / 2023 / 2025	Java 17, Java 21 and Java 25 became major LTS releases.

Why was Java developed?

- *To create software that can run on different types of computers.*
- *To support secure, reliable and network-based applications.*
- *To make programming easier through object-oriented concepts.*

WORA: Write Once, Run Anywhere. Java source code is compiled into bytecode, which can run on any compatible system having a JVM.

Quick recall

Developer — James Gosling | Original name — Oak | Source extension — .java | Bytecode extension — .class

JAVA FEATURES AND EDITIONS

2. Features of Java

Simple: Java syntax is familiar and avoids many complicated features such as pointer arithmetic.

Object-Oriented: Programs are organised using classes and objects. It supports inheritance, encapsulation and polymorphism.

Platform Independent: Compiled bytecode can run on different operating systems through a compatible JVM.

Secure: Java provides bytecode verification, class loading controls and runtime security features.

Robust: Exception handling and automatic memory management help make programs reliable.

Multithreaded: A program can perform multiple tasks concurrently using threads.

Portable: Java data types and bytecode rules support consistent behaviour across platforms.

High Performance: The Just-In-Time (JIT) compiler can convert frequently used bytecode into native machine code.

Distributed: Java provides networking libraries for communication between computers.

Dynamic: Classes and libraries can be loaded when a program runs.

3. Editions of Java

Edition	Main use	Examples / technologies
Java SE (Standard Edition)	Core Java and general-purpose applications.	Console programs, desktop apps, collections, JDBC, networking.
Java EE (Enterprise Edition)	Large-scale server-side and enterprise applications.	Web apps, enterprise services; evolved into Jakarta EE.
Java ME (Micro Edition)	Small or resource-limited devices and embedded systems.	Some embedded and connected-device applications.

Remember: SE = Standard/core Java; EE = Enterprise/server-side Java; ME = Micro/embedded Java.

Exam point

Java is not tied to one operating system. The JVM is platform-specific, while Java bytecode is designed to be portable.

JDK, JRE AND JVM

4. JDK vs JRE vs JVM

JVM (Java Virtual Machine) is the virtual machine that loads, verifies and executes Java bytecode. It also manages runtime memory and garbage collection.

JRE (Java Runtime Environment) provides the runtime components and libraries needed to run Java applications. In modern Java distributions, a separate JRE download is not always provided.

JDK (Java Development Kit) is used by developers to create Java applications. It includes the compiler, launcher and other development tools, along with runtime components.

Relationship (conceptual)

JDK	Development tools + runtime components
JRE	Runtime libraries + JVM (traditional model)
JVM	Loads and executes bytecode

Main differences

Point	JDK	JRE	JVM
Purpose	Develop, compile and run	Run applications	Execute bytecode
Compiler	Includes javac	Not its purpose	No
Main role	Complete development kit	Runtime environment	Execution engine

Easy memory trick

JDK = Development | JRE = Running environment | JVM = Bytecode execution

5. Source code, bytecode and execution

- *Source code: Human-readable Java instructions saved in a .java file.*
- *Bytecode: Intermediate instructions produced by javac and saved in a .class file.*
- *Execution: JVM loads the class and runs its bytecode, using interpretation and/or JIT compilation.*

Flow: Hello.java → javac → Hello.class → JVM → Output

```
Source file: Hello.java
| javac Hello.java
v
Bytecode file: Hello.class
| java Hello
v
JVM executes the program
```

JAVA COMMANDS AND CLASS STRUCTURE

6. Important Java commands

A. javac — Java compiler

It compiles source code (.java) into bytecode (.class).

```
javac Hello.java
```

B. java — Java launcher

It starts the Java application and runs the specified class using the JVM. Write the class name without the .class extension.

```
java Hello
```

C. jar — Java Archive tool

It packages class files, resources and metadata into a JAR archive. A JAR file is commonly used to distribute Java applications or libraries.

```
jar --create --file MyApp.jar Hello.class
java -jar MyApp.jar
```

7. Basic class structure and main()

A Java program is generally organised inside a class. The main() method is the traditional entry point of a basic Java application.

```
public class Hello {
public static void main(String[] args) {
System.out.println("Hello Java");
}
}
```

Meaning of main()

Part	Meaning
public	Accessible to the Java launcher.
static	Can be called without creating an object.
void	Returns no value.
main	Recognised entry-point method name.
String[] args	Receives command-line arguments.

Important rules

- *If a class is declared public, its source filename must match the public class name.*
- *Java is case-sensitive: Main and main are different.*
- *Statements usually end with a semicolon (;).*
- *The traditional main method uses the exact signature shown above.*

CLASSPATH, COMPILATION AND REVISION

8. Classpath

Classpath tells Java tools where to look for compiled classes and libraries required by a program. It may refer to folders, JAR files or other class-path entries.

```
java -cp . Hello
```

Here, -cp means classpath and . means the current directory.

9. Complete compilation and execution

Step 1: Create a file named Hello.java.

```
public class Hello {
public static void main(String[] args) {
System.out.println("Welcome to Java");
}
}
```

Step 2 — Compile:

```
javac Hello.java
```

This creates Hello.class if compilation succeeds.

Step 3 — Execute:

```
java Hello
```

Output:

```
Welcome to Java
```

10. One-page exam revision

Question / term	Answer to remember
Who developed Java?	James Gosling and his team.
Original name of Java?	Oak.
What is WORA?	Write Once, Run Anywhere.
Java source file?	Hello.java
Java bytecode file?	Hello.class
Compiler command?	javac
Run command?	java
Archive command?	jar
Bytecode execution engine?	JVM
Traditional entry point?	public static void main(String[] args)
Classpath?	Location(s) used to find classes and libraries.

Practice questions

- *Explain the history and evolution of Java.*
- *Write any five features of Java.*
- *Differentiate between JDK, JRE and JVM.*
- *Explain the Java execution process with a diagram.*

- *Explain javac, java, jar and classpath.*

Exam tip: Practise the JDK–JRE–JVM difference table, the Java execution flow and the main() method. These are useful for both theory and lab examinations.